Traversing a Learning-Progression Knowledge Graph to Diagnose Novice Code-Comprehension Reasoning

Mid-Term Progress Report

Pradhyumn Nair
Department of Computer Science, Georgia Southern University
Statesboro, GA, USA
pn02774@georgiasouthern.edu

ABSTRACT

A correct answer to a code-tracing question is weak evidence of correct reasoning, and a wrong answer rarely says which piece of understanding is missing. We study whether a knowledge graph that encodes a learning progression can make that diagnosis explicit and traceable. The graph covers Chapter 1 of an introductory Java textbook: 173 concepts joined by 241 prerequisite and parallel edges that each carry a written *edge logic*, 296 literature-sourced misconceptions and errors attached through 569 edges that pair the wrong rule with its correct twin, and five level nodes (L0–L4) per concept whose 3,079 "ways" state what a student at that level says. Our pipeline has two stages. Stage 1, implemented and tested, reads only the program text, matches recognition patterns stored on the concept nodes, and extracts the subgraph of concepts, edges, misconceptions and levels relevant to the item. Stage 2, now in design, adapts Think-on-Graph: a language model traverses that subgraph with beam search, pruning edges against the student's clauses, and learning-progression rules turn the matched edges into a level per concept and a root cause. On 10 edge-case scenarios and one reference case, Stage 1 extracted a subgraph for every item: 2 to 26 concepts (9.6 on average) with their relation edges, misconceptions and L0–L4 levels, cutting the misconception edges a reasoner must consider from 569 to 63 on average. All 208 tests of the extractor pass.

KEYWORDS

knowledge graph, learning progression, code comprehension, misconceptions, Think-on-Graph, large language models, computing education

1 INTRODUCTION

Reading a program is a different skill from writing one, and it develops in stages [16, 17]. Novices hold specific, repeatable wrong ideas about what the machine does: that a loop condition is watched continuously, that a variable holds an expression rather than a value, that a local variable starts at zero [7, 21, 28]. These gaps matter because a wrong mental model can still produce a right answer. On an open online course, about one third of students could not explain their own working code [14], and in a first-year Java course only 17% of students held a sound model of reference assignment [19].

Grading the output of a tracing question therefore measures the wrong thing. An instructor, or an automated tutor choosing the next exercise, needs to know *which* piece of understanding is missing and *how far along* the learner is. Computing education has scales of depth (SOLO [2], the Block Model [26]) and large catalogues of misconceptions [5, 24], and science education has *learning progressions*: ordered, validated descriptions of how reasoning about one idea develops [13, 30]. Our scoping review of 240 studies [20] found that these three ingredients (code-comprehension assessment, validated ordered scales, and language-model scoring) have each been paired, but not yet joined.

Large language models (LLMs) can now read free-text explanations and grade them at scale [8, 23, 31], but their verdicts are hard to trace to a rule, and LLM judges show known biases [32]. Recent work on reasoning over knowledge graphs offers a remedy: Think-on-Graph lets an LLM explore a graph step by step and answer from the paths it found [29], and Reasoning on Graphs restricts the model to paths that really exist in the graph [18].

This project joins the two lines. We built a knowledge graph that encodes a learning progression for Chapter 1 of Sedgewick and Wayne's *Introduction to Programming in Java* [27], with the reasoning written on every edge, and we are building a pipeline that traverses it to diagnose a student's explanation. The contributions reported at mid-term are:

- a learning-progression knowledge graph (KG) whose edges carry their logic, whose misconceptions carry a correct twin, and whose level nodes carry the ways a student at each level reasons (Section 4.1);
- a tested, data-driven extractor that reads only program text and returns the relevant subgraph, with every recognition pattern stored on the concept nodes rather than in code (Section 4.2);
- a design for adapting Think-on-Graph traversal and learning-progression rules to diagnosis, and preliminary evidence on 10 scenarios (Sections 4.3–5).

2 RESEARCH QUESTIONS

RQ1 *Subgraph recovery.* To what extent can the concepts, edges, misconceptions and level nodes relevant to a code-comprehension item be recovered from the program text alone, using recognition patterns stored

on the KG's concept nodes? *Measures:* size and connectedness of each subgraph, reduction of the search space, and the extractor's test results.

RQ2 *Traversal.* How accurately does a Think-on-Graph traversal of the extracted subgraph align each clause of a student's explanation with the edge or misconception it uses? *Measures:* following Hwang et al. [9], each edge and misconception of the subgraph is labelled *correct*, *incorrect* or *not applied*, and we report knowledge precision, knowledge recall and F1 against expert labels; exact-match accuracy of the held misconception and the root cause, as Hits@1 in ToG [29].

RQ3 *Learning-progression logic.* How well do the level rules stored in the graph place a student on L0–L4 for each concept, compared with expert labels? *Measures:* exact-match accuracy of the level, agreement with the acceptable set of levels, and weighted Cohen's κ.

RQ4 *Value of the graph.* Does grounding an LLM in the traversal improve diagnostic accuracy and run-to-run consistency over the same LLM without the graph? *Measures:* differences in the RQ2–RQ3 measures, agreement across five reruns, and LLM calls per answer.

RQ1 is answered with preliminary results in this report; RQ2–RQ4 define the remaining work.

3 LITERATURE REVIEW

We organise prior work around the three ingredients our system combines, and the method that joins them.

3.1 What novices get wrong, and how it is catalogued

Early work identified language-independent "bugs" in novice thinking [21] and the separate difficulties novices face at once: the notional machine, notation, standard structures and pragmatics [7]. Sorva's notional machines explain many of them [28], and Qian and Lehman's review [24] organises them into syntactic, conceptual and strategic knowledge. Misconceptions have since been identified through formal interviews with students, which revealed four themes [12], validated for Java toward a concept inventory [3], and curated into ProgMiscon, an online inventory of 198 misconceptions [5]. Learning-curve analysis separates the errors practice fixes from those that persist [25]. *Gap:* these catalogues are flat lists. They record that a misconception exists, but not which prerequisite relation it breaks or which level of understanding it implies, which is what a diagnosis needs.

3.2 Scales of depth for code comprehension

The SOLO taxonomy grades an answer by how far its parts are tied together [2], and was applied to novice code-reading responses [17], building on a multinational tracing study [16]. The Block Model grades the reading itself, by the size of the program unit and the depth from text to purpose [26]. Both order responses, not learners, and both depend on human raters whose agreement must be checked. Knowledge tracing models a learner's mastery over time, from Bayesian knowledge tracing [6] to deep knowledge tracing [22], but models correctness on items, not the reasoning behind an answer. *Gap:* no scale for code comprehension has been built from, and tested against, the specific wrong ideas students hold.

3.3 Learning progressions

Science education describes how reasoning about one idea develops as a learning progression: ordered levels drawn from real student answers [13]. Wilson's construct-modeling approach ties each level to assessment items [30]; Alonzo and Steedle built and tested a force-and-motion progression with ordered multiple-choice items whose options map to levels [1]; and Jin et al. set out the evidence a progression's validity requires [11]. Automated scoring against a progression is now feasible: machine scores can be compared directly with researchers' interpretations [10]. *Gap:* computing education has no validated progression for code comprehension, and existing progressions are documents for human raters, not machine-readable structures that a system can traverse.

3.4 Language models for grading and tutoring

Since 2022, LLMs have entered computing education as generators of material, explainers and tutors [23]. CodeHelp showed guarded LLM help at course scale [15], automatic grading of "explain in plain English" answers reached accuracy close to trained teaching assistants [8], and LLMs now classify student programming answers onto SOLO levels [31]. Using LLMs as judges has known weaknesses: position, verbosity and self-enhancement biases, and weak grading of math and reasoning [32]. *Gap:* these systems apply a general-purpose scale through a prompt. None constrains the model to a validated, ordered structure of the domain, and none returns the specific prerequisite that failed.

3.5 Reasoning over knowledge graphs with language models

Graph-based methods let an LLM reason over explicit structure rather than from memory. Think-on-Graph (ToG) treats the LLM as an agent that runs beam search on a knowledge graph. At each depth it lists the relations at the current entities, prunes them, follows the kept ones, and asks itself whether the paths found so far suffice [29]. It is training-free and set new best results on six of nine benchmarks with GPT-4. Reasoning on Graphs (RoG) instead plans relation paths, retrieves only the paths that exist in the graph, and reasons over them, which makes answers faithful and interpretable [18]. PathRAG shows that giving the LLM key relational paths as ordered text, rather than a flat list of nodes and edges, improves answer quality [4]. Principal-knowledge grounding checks reasoning against a set of required knowledge units and labels each correct, incorrect or not applied [9], but its units are a flat list without relations. *Gap:* these

Table 1: The learning-progression knowledge graph (v3.9.5-reader4).

Element	Count
Sections / topics / concepts	6 / 36 / 173
Prerequisite / parallel edges (with edge logic)	207 / 34
Misconceptions / errors	255 / 41
Misconception and error edges (wrong + twin rule)	569
Level nodes (L0–L4 per concept)	865
Level ways / ways linked to a misconception	3,079 / 736
Concepts with code patterns / regexes	166 / 359

methods answer factual questions over encyclopedic graphs. None has been applied to diagnosing a student's reasoning, where the graph encodes prerequisites, misconceptions and levels, and where the output is a level and a root cause, not an entity.

3.6 Positioning

Our work sits at the intersection the review identified [20]. We make the learning progression machine-readable as a knowledge graph, and use graph traversal, in the spirit of ToG and RoG, to tie an LLM's reading of a student explanation to explicit edges, misconceptions and levels.

4 METHODOLOGY

Figure 1 shows the pipeline. Stage 1 is implemented; Stage 2 is being built.

4.1 Dataset

Knowledge graph. The KG (v3.9.5-reader4, JSON; also exported as a spreadsheet) covers Chapter 1 of [27]: 6 sections, 36 topics and 173 concepts. Table 1 summarises it, and Figure 2 shows its schema. Each prerequisite or parallel edge carries an *edge logic*, a sentence stating why the relation holds; for example, `loop_control`→`for_loop`: "The for header is exactly the three control slots, and the body runs after the test and before the update on each pass." Each misconception edge carries the wrong rule (`logic`) and the correct rule (`twin_logic`). 294 of the 296 misconception and error nodes cite literature, and their provenance is published work (167), the textbook (92), derived (28), a rule dictionary (7) or the scenarios (2). Every concept has five level nodes, L0–L4, aligned with SOLO-style depth: L0 holds a rule that contradicts Java; L1 names the construct or states its rule; L2 makes a correct prediction or trace; L3 relates the concept to a neighbour; L4 diagnoses an error or predicts the effect of a change. Each level node is reached by a `has_level` edge listing its *ways*, 3,079 in total; 736 of them link a specific misconception, so a held misconception maps directly to a level. Each concept also has a *reader* block with recognition patterns; 166 concepts are visible in code (359 regular expressions), 7 are not (for example `testing` and `flowchart`), and 21 are marked generic (for example `statement`).

Scenarios. We use the scenario file in the repository's `data/raw` folder: 10 edge-case scenarios (S001, S007, S008, S021, S026, S047, S052, S057, S134, S137) and one reference case (S000), drawn from a Chapter 1 scenario set; each has code, a question and a student answer. They were chosen as hard cases: self-corrections, correct answers by coincidence, non-native English. Stage 1 reads only the code; the question and answer are inputs to Stage 2.

Preprocessing. String and character literals and comments are blanked to spaces, keeping the length so positions and line numbers stay valid. The code is split into statements on semicolons and braces only at parenthesis depth zero, so a `for` header stays whole. Shell-command fragments (`java X < file`) are extracted for the I/O concepts. The student answer is split into clauses at link words stored in the KG (*because*, *so*, *but*, *oh wait*, ...); this step belongs to Stage 2.

4.2 Stage 1: subgraph extraction (implemented)

The extractor is data-driven. It loops over the concept nodes and runs each node's own patterns against the code: regular expressions over the whole program, structural signals over the program or over each statement, operator symbols, Java type names, and command patterns. A concept is tagged only when one of its own patterns matches, and the code contains no concept identifiers. Generic concepts are kept only when a prerequisite or parallel edge ties them to a specific tagged concept, or a decisive signal of theirs fired. The subgraph is then read from the KG: every relation edge with both ends among the concepts, every misconception and error edge from them, every `has_level` edge with its ways, and the level–misconception links inside the ways. Figure 3 shows the result for scenario S007.

Stage 1 on S007, step by step. The S007 code is a `for` loop over `i` from 3 down to 0, with an inner `for` loop that prints `j` with `printf` and a `println` after each outer pass. Table 2 lists what tagged each specific concept. Every tag comes from a pattern stored on that concept's own node. Two are structural signals rather than plain patterns: `loop_control` is tagged by its `for_header` signal, which needs the whole header `(int i = 3; i >= 0; i--)`, and `nested_loops` by its `nested_for` signal, which finds a second `for` inside the first loop's braces. Five generic concepts are kept because an edge ties each to a specific one (for example `string_literal`→`println`). The KG then supplies 14 relation edges among the 17 concepts, including `loop_control`→`for_loop` and `for_loop`→`nested_loops`, and the concepts fall into three pieces: the loop concepts, the output concepts, and `loop_body` on its own.

4.3 Stage 2: Think-on-Graph traversal and learning-progression logic (design)

We adapt ToG [29] from question answering to diagnosis, keeping its three phases. *Initialization:* the main concepts of the item are the topic entities. *Exploration:* at each depth,

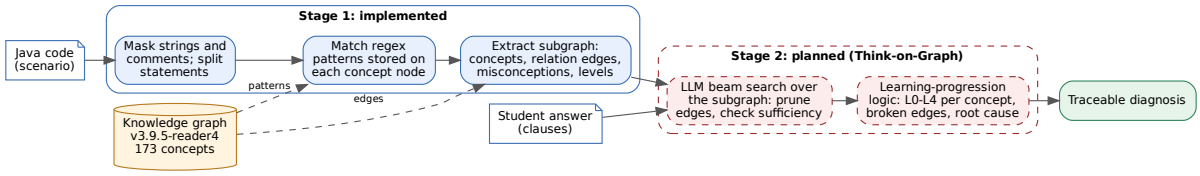


Figure 1: Pipeline. Stage 1 (implemented) reads only the code and extracts the relevant subgraph using patterns stored on the concept nodes. Stage 2 (planned) adapts Think-on-Graph to traverse that subgraph against the student's answer and applies the learning-progression rules.

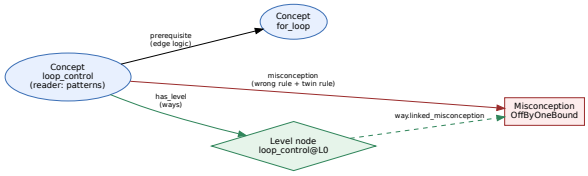


Figure 2: KG schema. A held misconception reaches a level through the way that names it.

Table 2: What tagged each specific concept in S007 (line of the code where the match starts).

Concept	Tagged by	Line
`for_loop`	pattern `for (`	1
`loop_control`	signal `for_header`	1
`nested_loops`	signal `nested_for`	1
`loop_variable`, `loop_scope`	patterns on the `for` header	1
`loop_body`, `control_flow`	pattern `for ( / if ( ...`	1
`boolean_expression`	pattern for a `for` test	1
`comparison_operators`	pattern `>=`, `>`	1
`increment_operators`	pattern `--`	1
`formatted_output`	pattern `.printf(`	3
`println`	pattern `System.out.println(`	5

relation search lists the edges at the frontier within the subgraph (relation edges with their logic, misconception edges with wrong and twin rules, and level ways), and relation pruning asks the LLM which of them the current clause uses, keeping the top N; entity exploration follows the kept edges. *Reasoning:* the LLM is asked whether the paths found are sufficient to judge the clause; if not, exploration continues to depth $D_{\max}$. Following ToG, $N = D_{\max} = 3$, which bounds the cost at $2ND + D + 1 = 22$ LLM calls per clause. Paths are given to the LLM as ordered text, node, edge logic, node, as PathRAG recommends [4], and the LLM sees only edges that exist, as in RoG [18]. Figure 4 sketches the intended traversal for S007.

The learning-progression logic then turns matched edges into levels, using only rules stored in the KG. (i) A clause closer to a misconception's wrong rule than to its twin, and confirmed by a fact from the code (for S007, the last pass is $i = 0$), holds that misconception; the level is the L0 way that links it. (ii) A relation edge whose logic the clause states correctly is confirmed and supports L3 for its ends; an edge whose end holds a misconception is broken. (iii) Otherwise the level is the highest whose ways the clause matches, using the KG's evidence classes (named and rule: L1, run: L2, relation: L3, diagnosis and hypothetical: L4). (iv) The root cause is found by walking up prerequisite edges from the main concept to the first broken edge.

Worked example (design). In S007 the student writes "3 2 1 / 2 1 / 1 (no empty line at the end), because the loop stops at 1." The clause splitter yields c1 (the output claim) and c2 (the reason, after *because*). Starting from `nested_loops`, the traversal for c1 keeps `for_loop`→`nested_loops`, whose logic ("the inner for runs its complete set of passes for every single pass of the outer for") matches the three rows; that edge is confirmed. For c2 it continues to `loop_control`→`for_loop` and then to the misconception edge `loop_control`→OffByOneBound, where the clause is closer to the wrong rule than to the twin. The fact read from the loop header (test `i >= 0`, so the last pass is $i = 0$) confirms that the student is wrong. The rules then give `loop_control` L0 through its linked way, mark `loop_control`→`for_loop` broken, give `nested_loops` L0 because its output claim is contradicted, and name `loop_control` as the root cause. This is the diagnosis the implementation will be tested against.

4.4 Training procedure

Stage 1 needs no training: the patterns are written into the KG and tested. Stage 2 follows ToG in being training-free: the LLM is used through prompts with a few worked examples, temperature 0 for the reasoning step, and no fine-tuning. If prompting proves unreliable on near-identical wrong and twin rules, we will fine-tune a small open model on pairs generated from the KG itself (wrong rule, twin rule, edge logic and level ways as labelled statements), as RoG builds its training data from graph paths [18]; the 10 scenarios will be held out for testing only. Hyperparameters are fixed in advance: $N = 3$, $D_{\max} = 3$, at most 800 characters between a declaration and its use in bounded patterns. Stage 1 runs

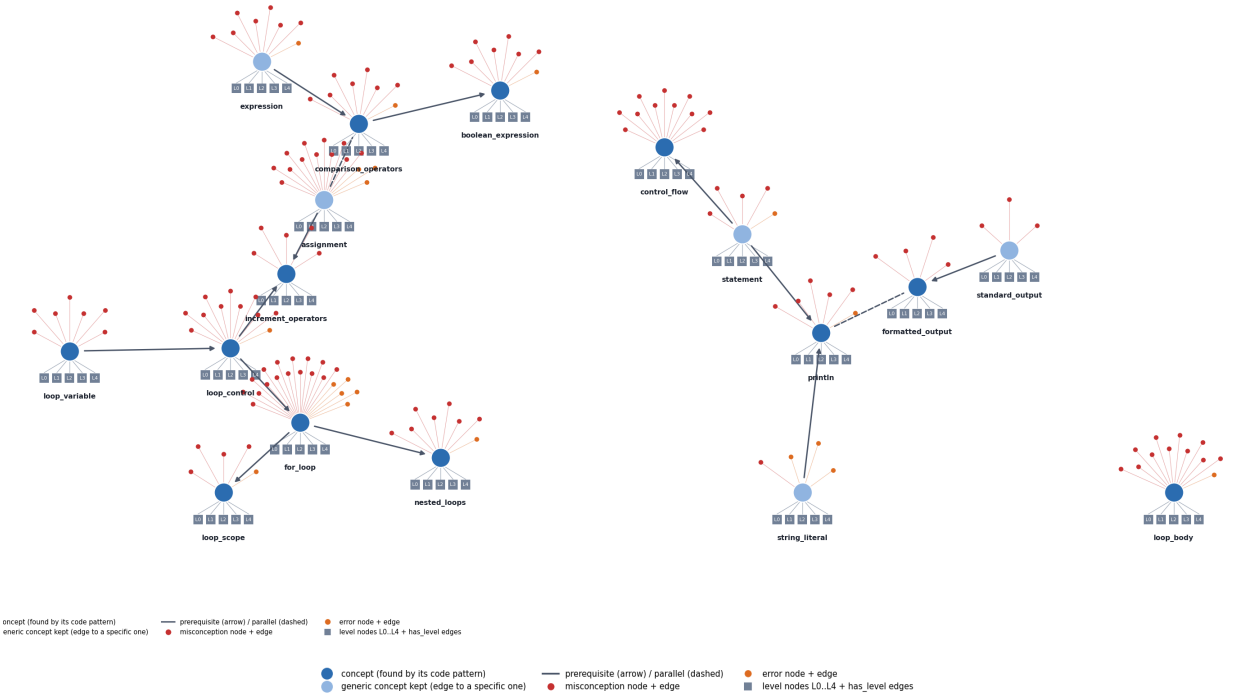


Figure 3: Stage 1 subgraph for S007 (nested loops; the student says "the loop stops at 1"), produced by the repository's results script: 17 concept nodes, 14 relation edges, 141 misconception and error edges, 85 level nodes, in three disconnected pieces. Dark nodes were found by their own code patterns; light nodes are generic concepts kept because an edge ties them to a specific one.

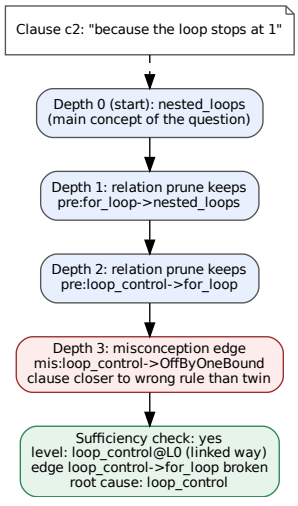


Figure 4: Intended Think-on-Graph traversal for clause c2 of S007 (design illustration, not a result).

on a laptop CPU; Stage 2 calls a hosted LLM through its API.

4.5 Evaluation

Stage 1 is evaluated by its test suite and by the subgraphs it produces (RQ1). The test suite checks 372 snippets, showing that each of the 166 code-visible concepts is tagged when it should be and not when it should not, plus core-rule tests that patterns come only from the nodes. Stage 2 will be evaluated in two conditions: on the Stage 1 subgraph, and on a subgraph built from expert-chosen concepts, which separates traversal errors from extraction errors. The baselines are (B1) the same LLM given the code, answer and level definitions but no graph; (B2) a RoG-style single call given the subgraph paths without iterative traversal; and (B3) a lexical TF-IDF judge in place of the LLM. Ablations remove the prerequisite edges, the twin rules or the level ways. Consistency is measured over five reruns.

Output of a diagnosis. For each student answer the system returns the overall level, a level per concept, the misconceptions held, the root-cause concept, and for each verdict the evidence path in the KG (for S007: `loop_control`→OffByOneBound→`loop_control@L0`). An answer is correct under exact match only if the predicted level, misconception or root cause equals the expert label.

Table 3: RQ1: Stage 1 subgraphs over the 10 scenarios (S000 reference case excluded).

Per subgraph	Mean	Min	Max
Concept nodes	9.6	2	26
Generic concepts kept	2.5	1	5
Relation edges	8.2	1	22
Misconception and error edges	63.0	7	167
Level nodes	48.0	10	130
Level–misconception links	69.0	11	193
Disconnected pieces	2.0	1	4

Result metric. Hwang et al. assess a model's reasoning against the knowledge units a question requires, labelling each unit *correct*, *incorrect* or *not applied*, and report knowledge precision (of the units used, the share used correctly), knowledge recall (of the units required, the share used correctly) and their F1 [9]. We adopt this metric with the edges and misconceptions of the subgraph as the knowledge units: a confirmed edge is *correct*, a broken edge or a held misconception is *incorrect*, and an edge the student never uses is *not applied*. For S007, `for_loop`→`nested_loops` is correct and `loop_control`→`for_loop` is incorrect through OffByOneBound. Precision, recall and F1 are computed between the system's labels and the expert's labels for each scenario.

Validating an LLM judge. Expert labels are the reference. Where an LLM labels units at scale, we follow Zheng et al., who validate an LLM judge by its agreement with human judgments and document its position, verbosity and self-enhancement biases [32]. We report the LLM judge's agreement with the expert labels (percent agreement and Cohen's κ) on the 10 scenarios before using its labels, shuffle the order in which edges are presented, use temperature 0, and use a different model family for judging than for reasoning.

5 RESULTS AND FINDINGS SO FAR

Stage 1 extracts a subgraph for every scenario. Table 3 and Table 4 report RQ1. For all 10 scenarios and the reference case, the extractor found concept nodes from the code and read their relation edges, misconceptions and level nodes out of the KG. Subgraphs range from 2 to 26 concepts (9.6 on average, Figure 5). The repository's report prints "fully covered: 10 / 10", and all 208 tests pass in 2.3 seconds.

The graph narrows the search. A subgraph holds 63 of the KG's 569 misconception and error edges on average (Figure 6), about 11% of the graph, so a reasoner works with a small, relevant set rather than the whole catalogue. Scenarios with short code (S026, S047, S134) give subgraphs of two concepts; longer code with loops, arrays and output (S007, S008, S057) gives 17 to 26 concepts.

Larger subgraphs split into pieces. Five of the ten subgraphs have more than one connected piece (up to four), because the KG has no edge between some constructs, for

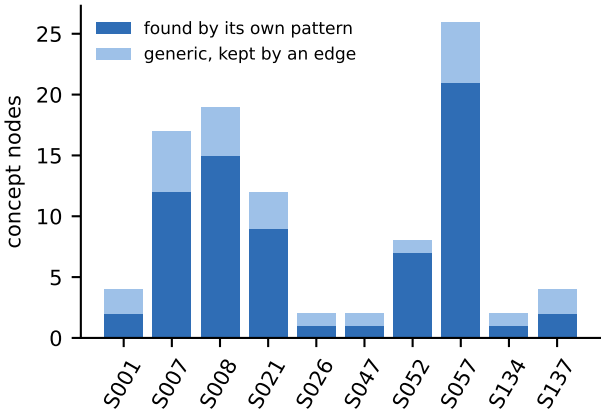


Figure 5: Concept nodes in each Stage 1 subgraph: found by their own pattern (dark) and generic concepts kept because an edge ties them to a specific one (light).

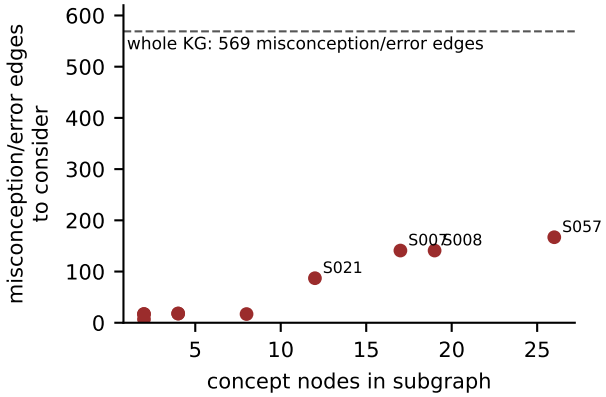


Figure 6: Misconception and error edges in each subgraph, against the 569 in the whole KG.

example between output formatting and loop control in S007. A traversal therefore needs more than one starting point.

The learning-progression links are in the subgraph. Every subgraph carries its concepts' level nodes and the links from level ways to misconceptions (69 per subgraph on average), so the rules that turn a matched misconception into a level are available to Stage 2 without another lookup. For example, S007's subgraph contains the misconception edge `loop_control`→OffByOneBound and the way that links it to `loop_control@L0`.

Limitations. The scenarios are 10 hand-picked edge cases and one reference case, not a random sample, so the results show feasibility, not accuracy. They were kept because their code shows the concepts the items are about; items whose concepts appear only in the student's answer are left to

Table 4: Stage 1 per scenario: concept nodes (generic kept), relation edges, misconception and error edges, misconception nodes, level nodes, level–misconception links and disconnected pieces.

ID	Concepts (generic)	Relation edges	Mis. edges	Mis. nodes	Level nodes	Links	Pieces
S000 (ref.)	4 (2)	2	18	17	20	20	2
S001	4 (2)	5	18	15	20	16	1
S007	17 (5)	14	141	104	85	145	3
S008	19 (4)	16	141	108	95	161	4
S021	12 (3)	10	87	69	60	91	2
S026	2 (1)	1	17	15	10	17	1
S047	2 (1)	1	7	6	10	11	1
S052	8 (1)	7	17	16	40	23	2
S057	26 (5)	22	167	120	130	193	4
S134	2 (1)	1	17	15	10	17	1
S137	4 (2)	5	18	15	20	16	1

Stage 2. The KG covers one chapter of one textbook, and the reasoning stage is not built yet, so RQ2–RQ4 have no results.

6 DISCUSSION: WHAT STAGE 1 IMPLIES FOR THE TRAVERSAL

The preliminary results shape the traversal design in four ways.

Start from the question, not from the whole subgraph. ToG starts from topic entities named in the question [29]. A subgraph holds every construct in the code, up to 26 concepts. The traversal will start from the item's main concepts and treat the rest of the subgraph as reachable context.

Allow several starting points. Subgraphs with more than one piece (Table 4) cannot be covered by a walk from one entity. Following ToG, the beam will hold up to N paths from different starting entities, so that an output claim about formatting and a reason about the loop bound are explored in parallel.

Add the student's words as a second entry point. A student can name a concept the code does not show, for example a construct they compare with the one in the code. Before traversal, concepts named in the answer will be added through the aliases stored on the concept nodes, which the extractor deliberately ignores. This keeps Stage 1 code-only and testable while giving the traversal the concepts it needs.

Cost. With the ToG bound of 22 LLM calls per clause and the two to three clauses of a typical answer, a scenario costs at most about 66 calls, against one call for an LLM without the graph. RQ4 asks whether the gain in accuracy and consistency justifies that cost; a RoG-style single call over the extracted paths (B2) is the cheaper alternative we will compare against.

7 REMAINING WORK AND TIMELINE

Table 5 gives the plan to the end of the semester.

Table 5: Week-by-week plan (Fall 2026).

Week	Milestone
Nov 2–8	Clause segmentation; ToG prompts; S007 end to end.
Nov 9–15	ToG traversal over Stage 1 and expert-chosen subgraphs; logging of paths and LLM calls.
Nov 16–22	Learning-progression rules and root cause; RQ2 and RQ3 metrics; baseline B1 (LLM without graph).
Nov 23–29	Baselines B2 (RoG-style) and B3 (lexical); ablations; five reruns for consistency (RQ4).
Nov 30–Dec 6	Second annotator (κ); error analysis; answer-side concept retrieval.
Dec 7–13	Final paper, slides and repository release.

Risks and contingencies. LLM access or cost: the traversal is model-agnostic; if API access is limited we run an open model locally and report the lexical judge (B3) as a floor. *Small dataset:* we report per-scenario results and bootstrap confidence intervals, and frame the study as a case study; collecting further student answers is planned beyond this semester. *Label validity:* a second annotator labels the 10 scenarios and we report agreement. *Non-determinism:* temperature 0 and five reruns per scenario. *Concepts outside the code:* retrieve concepts from the question and answer through the aliases stored in the KG before traversal.

ACKNOWLEDGMENTS

I thank my advisor, Dr. Vijayalakshmi Ramasamy, for guidance. Generative AI disclosure: Claude (Anthropic) was used to help draft and edit this report, generate figures from the measured results, and assist with code and pattern development in the repository; all results were produced by running the repository's code, and all content was reviewed by the author.

REFERENCES

[1] Alicia C. Alonzo and Jeffrey T. Steedle. 2009. Developing and Assessing a Force and Motion Learning Progression. *Science Education* 93, 3 (2009), 389–421.

[2] John B. Biggs and Kevin F. Collis. 1982. *Evaluating the Quality of Learning: The SOLO Taxonomy (Structure of the Observed Learning Outcome)*. Academic Press, New York.

[3] Ricardo Caceffo, Pablo Frank-Bolton, Renan Souza, and Rodolfo Azevedo. 2019. Identifying and Validating Java Misconceptions Toward a CS1 Concept Inventory. In *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '19)*. 23–29.

[4] Boyu Chen, Zirui Guo, Zidan Yang, Yuluo Chen, Junze Chen, Zhenghao Liu, Chuan Shi, and Cheng Yang. 2026. PathRAG: Pruning Graph-Based Retrieval Augmented Generation with Relational Paths. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[5] Luca Chiodini, Igor Moreno Santos, Andrea Gallidabino, Anya Tafliovich, André L. Santos, and Matthias Hauswirth. 2021. A Curated Inventory of Programming Language Misconceptions. In *Proceedings of the 26th ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '21) V.1*. 380–386.

[6] Albert T. Corbett and John R. Anderson. 1994. Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge. *User Modeling and User-Adapted Interaction* 4, 4 (1994), 253–278.

[7] Benedict du Boulay. 1986. Some Difficulties of Learning to Program. *Journal of Educational Computing Research* 2, 1 (1986), 57–73.

[8] Max Fowler, Binglin Chen, Sushmita Azad, Matthew West, and Craig Zilles. 2021. Autograding "Explain in Plain English" Questions Using NLP. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education (SIGCSE '21)*. 1163–1169.

[9] Hyeon Hwang, Yewon Cho, Chanwoong Yoon, Yein Park, Minju Song, Kyungjae Lee, Gangwoo Kim, and Jaewoo Kang. 2025. Assessing LLM Reasoning Steps via Principal Knowledge Grounding. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, Suzhou, China, 19925–19948. https://aclanthology.org/2025.findings-emnlp.1085/

[10] Hui Jin, Cynthia Lima, and Limin Wang. 2025. Automated Scoring in Learning Progression-Based Assessment: A Comparison of Researcher and Machine Interpretations. *Educational Measurement: Issues and Practice* 44, 3 (2025), 25–37.

[11] Hui Jin, Peter van Rijn, John C. Moore, Malcolm I. Bauer, Yamina Pressler, and Nissa Yestness. 2019. A Validation Framework for Science Learning Progression Research. *International Journal of Science Education* 41, 10 (2019), 1324–1346.

[12] Lisa C. Kaczmarczyk, Elizabeth R. Petrick, J. Philip East, and Geoffrey L. Herman. 2010. Identifying Student Misconceptions of Programming. In *Proceedings of the 41st ACM Technical Symposium on Computer Science Education (SIGCSE '10)*. 107–111.

[13] Joseph Krajcik and Namsoo Shin. 2023. Student Conceptions, Conceptual Change, and Learning Progressions. In *Handbook of Research on Science Education, Volume III*. Routledge, New York, NY, 121–157.

[14] Teemu Lehtinen, Aleksi Lukkarinen, and Lassi Haaranen. 2021. Students Struggle to Explain Their Own Program Code. In *Proceedings of the 26th ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '21) V.1*. 206–212.

[15] Mark Liffiton, Brad E. Sheese, Jaromir Savelka, and Paul Denny. 2023. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. 1–11.

[16] Raymond Lister, Elizabeth S. Adams, Sue Fitzgerald, William Fone, John Hamer, Morten Lindholm, Robert McCartney, Jan Erik Moström, Kate Sanders, Otto Seppälä, Beth Simon, and Lynda Thomas. 2004. A Multi-National Study of Reading and Tracing Skills in Novice Programmers. *ACM SIGCSE Bulletin* 36, 4 (2004), 119–150.

[17] Raymond Lister, Beth Simon, Errol Thompson, Jacqueline L. Whalley, and Christine Prasad. 2006. Not Seeing the Forest for the Trees: Novice Programmers and the SOLO Taxonomy. In *Proceedings of the 11th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE '06)*. 118–122.

[18] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. 2024. Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning. In *International Conference on Learning Representations (ICLR)*.

[19] Linxiao Ma, John Ferguson, Marc Roper, and Murray Wood. 2007. Investigating the Viability of Mental Models Held by Novice Programmers. In *Proceedings of the 38th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '07)*. 499–503.

[20] Pradhyumn Nair and Vijayalakshmi Ramasamy. 2026. Learning Progressions and Large Language Models for Assessing Code Comprehension: A Scoping Review. (2026). Manuscript under review.

[21] Roy D. Pea. 1986. Language-Independent Conceptual "Bugs" in Novice Programming. *Journal of Educational Computing Research* 2, 1 (1986), 25–36.

[22] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas, and Jascha Sohl-Dickstein. 2015. Deep Knowledge Tracing. In *Advances in Neural Information Processing Systems 28*. 505–513.

[23] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. 2023. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '23)*. 108–159.

[24] Yizhou Qian and James Lehman. 2017. Students' Misconceptions and Other Difficulties in Introductory Programming: A Literature Review. *ACM Transactions on Computing Education* 18, 1 (2017), 1:1–1:24.

[25] Kelly Rivers, Erik Harpstead, and Ken Koedinger. 2016. Learning Curve Analysis for Programming: Which Concepts Do Students Struggle With?. In *Proceedings of the 2016 ACM Conference on International Computing Education Research (ICER '16)*. 143–151.

[26] Carsten Schulte. 2008. Block Model: An Educational Model of Program Comprehension as a Tool for a Scholarly Approach to Teaching. In *Proceedings of the 4th International Workshop on Computing Education Research (ICER '08)*. 149–160.

[27] Robert Sedgewick and Kevin Wayne. 2017. *Introduction to Programming in Java: An Interdisciplinary Approach* (2nd ed.). Addison-Wesley, Boston, MA.

[28] Juha Sorva. 2013. Notional Machines and Introductory Programming Education. *ACM Transactions on Computing Education* 13, 2 (2013), 8:1–8:31.

[29] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *International Conference on Learning Representations (ICLR)*.

[30] Mark Wilson. 2009. Measuring Progressions: Assessment Structures Underlying a Learning Progression. *Journal of Research in Science Teaching* 46, 6 (2009), 716–730.

[31] Shan Zhang, Pragati Shuddhodhan Meshram, Priyadharshini Ganapathy Prasad, Maya Israel, and Suma Bhat. 2025. An LLM-Based Framework for Simulating, Classifying, and Correcting Students' Programming Knowledge with the SOLO Taxonomy. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education (SIGCSE TS 2025) V.2*. 1681–1682.

[32] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*. 46595–46623.